

P2P Intercom

Technical manual for `server.py` and `clint.py` — a two-party voice intercom over UDP, with a signalling server that matches the peers and reports the remote client's headphone state.

Component	Role
<code>server.py</code>	Tkinter GUI. Hosts the optional internal signalling server, runs one side of the call, shows a live dashboard of the link, beeps when the remote headphones connect, and can shut the remote client down.
<code>clint.py</code>	Headless console client. Registers with the server, reports its headphone state once per second, runs the other side of the call. No window, no keyboard input.

This manual is written in English to match the interface: every button label, log line and setting key it refers to appears verbatim on screen.

1. How a call is established

Both sides speak to a signalling server only long enough to learn each other's address. The audio itself never passes through the server — it goes straight between the two machines.

Step by step

1. **Register.** Each side sends `{"id": "..."}` to the signalling server once per second and the server answers `{"ack": true}`.
2. **Match.** As soon as two different IDs are registered, the server sends each one the other's IP, port and ID, five times, then clears the pool.
3. **Punch.** Each side fires tagged PUNCH datagrams straight at the peer for two seconds. This opens the NAT mapping in both directions.
4. **Talk.** Both sides open their microphone and speaker and exchange tagged audio datagrams peer to peer.

Why the tag byte matters. Signalling is JSON and starts with `{`. P2P packets carry a one-byte tag instead (`\x01` audio, `\x02` punch). Both sockets receive both kinds, and without the tag a JSON control message would be handed to the speaker and played as noise.

Address locking

The server reports the address it saw, but a NAT may hand the peer a different source port. Each side therefore locks onto the address the first tagged packet actually arrived from, and from then on ignores tagged packets from anywhere else. That covers the NAT case and also stops anyone else on the network from injecting audio into a live call.

Limit. Hole punching works with cone NAT. Symmetric NAT rewrites the port per destination, so the punched mapping does not match and the call will not connect. That needs a relay (TURN), which this project does not implement.

2. Installing and running

Requirements

```
pip install sounddevice numpy

python server.py      # the GUI machine
python clint.py       # the headless machine
```

Tkinter ships with Python on Windows. winsound is part of the standard library on Windows only; on Linux and macOS the import is guarded and the app simply runs without the beep.

Testing both sides on one computer

Two machines are not required to exercise everything except real audio:

- Run `server.py` and press **START INTERNAL SERVER**.
- Set the client's `settings.txt` to `"ip": "127.0.0.1"`.
- Give the two sides **different** IDs — for example `node_A` in the GUI and `node_B` in the file. Matching needs two distinct IDs.
- Run `clint.py` in its own console window.

Headphone detection, the beep, the dashboard and the shutdown button all work this way. A real call does not, because both processes would fight over the same sound card.

3. Configuration files

Both files are JSON, are created with defaults on first run, and are read again before every call attempt — so an edit takes effect without a restart.

settings_A.txt — server

Key	Meaning
<code>ext_ip</code>	Public address to report instead of a private one. Only relevant when the signalling server is not itself on the public internet.
<code>local_mode</code>	true keeps addresses exactly as seen and ignores <code>ext_ip</code> . Use this on a single LAN.
<code>ip / port</code>	Signalling server this GUI registers with. Point it at 127.0.0.1 when hosting the internal server.
<code>my_id</code>	This side's identifier. Must differ from the client's.

ext_ip does not translate the port. Only the address is rewritten, so this path needs a static port forward on the router. On one LAN, tick **Use Local Mode** instead.

settings.txt — client

Key	Meaning
<code>ip / port</code>	Signalling server address. Also the only address the client will accept a shutdown command from.
<code>my_id</code>	This client's identifier.
<code>hp_device</code>	Part of the headset's device name, matched case-insensitively. This is what makes unplug detection work — see section 7.

```
{
  "ip": "192.168.1.12",
  "port": "9999",
  "my_id": "node_B",
  "hp_device": "2- USB PnP"
```

```
}
```

4. Server window reference

Control	What it does
Help	Opens the in-app help, drawn in the application theme.
Quit	Ends the call, stops the internal server, closes the window.
External IP / Use Local Mode	Address rewriting policy for the internal signalling server. See section 3.
START INTERNAL SERVER	Binds UDP on the configured port and starts matching peers. The label only changes to STOP once the bind actually succeeded; a failure is reported on the SERVER CONNECTION row.
Server IP / Port / My ID	Which signalling server this side registers with, and under what name. Locked while a call is running.
START INTERCOM	Registers, waits for a peer, punches, then streams. Also silences the beep.
SHUTDOWN CLIENT	Asks the most recently heard client to exit its process. See section 6.

Dashboard rows

Three cards refresh five times a second. The worker threads only write plain values; a single timer redraws them, so no background thread ever touches a widget.

Row	Colour	Meaning
SERVER CONNECTION	grey	Not hosting and not registered.
	green	Internal server listening, or registered with an external one.
	amber	Registration sent, no peer assigned yet.
	red	Bind failed — the port is taken or blocked.
CLIENT CONNECTION	grey	No call running.
	amber	Waiting for a peer, punching, or peer audio has stopped.
	green	Audio arriving from the locked peer. Shows peer, address and call time.
CLIENT HEADPHONES	grey	Nobody has reported, or no report for over 5 seconds.
	green	Remote headphones present. Beeping until acknowledged.
	red	Remote headphones gone.

A stale row shows *no report for N s* in grey rather than staying green. A green light that is really just a stale memory is worse than an honest “unknown”.

5. Client console output

The client has no window. Every line below is a state change — it is not printed again while the state holds, so a quiet console means nothing has changed.

Line	Meaning
Audio devices detected: ...	Printed once at startup. Copy part of your headset's name into hp_device.
HEADPHONES: CONNECTED	The tracked headset now provides both a microphone and a speaker. Names and indexes follow. Reported to the server immediately.
HEADPHONES: DISCONNECTED	The tracked headset is gone. Any active call ends.

Line	Meaning
SERVER: CONNECTED	The server answered a heartbeat.
SERVER: DISCONNECTED	No answer for 3 seconds. The client keeps trying.
hp_device "x" matches no mic+speaker pair	The keyword is wrong or the device is unplugged. Printed once, not per poll.
Peer Target Assigned ...	The server matched this client with a peer.
Peer reached us from ...	The peer's real source address differs from the one reported. Normal behind NAT.
Peer disconnected (3 seconds timeout).	No audio for 3 seconds. The call ends and the client waits for a new match.
Shutdown command received from server.	A valid remote shutdown. The process exits.
Ignored shutdown command from ...	A shutdown arrived from something other than the signalling server, and was dropped.

6. Beep and remote shutdown

Beep on headphone connect

When the server sees a client's headphone state *change* to connected, it starts a repeating 1000 Hz beep, 200 ms long, every 1.5 seconds. It fires on the transition only — the client heartbeats once a second, and reacting to every report would beep forever.

It stops when you click anywhere on the CLIENT HEADPHONES card, when the headphones are unplugged, when you press START INTERCOM, when the internal server is stopped, and on close. While it rings, the card reads *click to silence*.

`winsound.Beep` blocks for the length of the beep, which is why the beep is kept short: silencing waits at most one beep. Raising `RING_MS` above roughly 400 makes the silence button feel sluggish.

SHUTDOWN CLIENT

The button targets the client that reported most recently, names it in a confirmation dialog, and on confirmation sends `{"cmd": "shut down"}` three times. The client ends any call, reports its disconnect and exits its process.

Two guards, both deliberate:

- **The command leaves the internal server's socket.** If that server is not running the button explains why instead of sending into the void — the client would reject it anyway.
- **The client obeys only its own signalling server.** Source address and port must both match the server it registered with. Anything else is logged and dropped, so a single stray datagram cannot kill a remote client.

This is one-way. The client exits its process. If it runs on a machine with no screen, restarting it needs physical access.

7. Troubleshooting

Headphone state never changes on the server

Almost always `hp_device` is empty. Without it the client watches the system *default* device, and Windows falls back to the built-in audio the moment a USB headset is unplugged — so the check keeps passing and the client keeps reporting “connected”.

Run the client, read the device list it prints, and put part of your headset's name in `hp_device`.

Be specific enough. If two USB dongles are present, “USB PnP” matches both, and unplugging one still leaves a match — the disconnect is swallowed. Use the distinguishing part, for example “2- USB PnP”.

Client says SERVER: DISCONNECTED

The server is not answering heartbeats. Check that the internal server is started, that the IP and port match on both sides, and that the port is open in the firewall. An old build of `server.py` does not send acknowledgements at all — both files must come from the same version.

Two sides never match

The two IDs must differ; the server matches the first two *distinct* IDs it sees. A registration older than 30 seconds is discarded, so a side that stopped registering will not be matched.

Audio connects, then drops after three seconds

Datagrams are not arriving from the peer. Usually symmetric NAT, or a firewall blocking the punched port. Check whether the log shows the peer reaching you from an unexpected address.

A widget disappeared after editing the server window

The dashboard is packed with `expand=True` and consumes whatever space is left. Anything packed *after* it gets none and is silently not drawn — no error, no warning. Pack new widgets **before** `_create_dashboard()`.

8. Wire protocol

Every datagram is either JSON signalling or a tagged P2P packet. Nothing else is accepted.

Client to server

Message	When	Effect
<code>{"id": X}</code>	Once a second while waiting for a peer	Enters the matching pool. Two distinct IDs trigger a match.
<code>{"id": X, "status": "hp", "headphones": bool}</code>	Once a second, always	Updates the dashboard and the beep. Never enters the matching pool.
<code>{"id": X, "status": "disconnected"}</code>	On exit, three times	Frees the slot and marks the client disconnected.

Server to client

Message	When	Effect
<code>{"ack": true}</code>	For every valid signalling message	Proves the server is alive. Three seconds without one flips the client to SERVER: DISCONNECTED.
<code>{"peer_ip": ..., "peer_port": ..., "peer_id": ...}</code>	On a match, five times	The client punches this address and starts streaming.
<code>{"cmd": "shutdown"}</code>	SHUTDOWN CLIENT, three times	The client exits — only if the datagram came from its own signalling server.

Peer to peer

Packet	Meaning
<code>\x01 + PCM</code>	Audio. 16 kHz, mono, signed 16-bit little endian, 512 frames.
<code>\x02PUNCH</code>	NAT hole punch. Opens the mapping; never played.

9. Tuning constants

All at the top of each file.

Shared audio

Constant	Value	Effect of changing it
SAMPLE_RATE	16000	Voice quality against bandwidth. Must match on both sides.
CHANNELS	1	Mono. Must match on both sides.
CHUNK_SIZE	512	Frames per packet: ~32 ms. Lower means less latency, more packets.
TIMEOUT_SECS	3.0	Silence from the peer before the call is dropped.

Server

Constant	Value	Effect of changing it
CLIENT_TTL	30.0	How long a registration stays eligible for matching.
MATCH_RETRIES	5	Copies of the peer-match reply. UDP may drop some.

Constant	Value	Effect of changing it
CMD_RETRIES	3	Copies of the shutdown command.
PUNCH_DURATION	2.0	Seconds of punching before streaming starts.
UI_TICK_MS	200	Dashboard refresh interval.
PEER_IDLE	1.5	Silence before the client row stops being green.
HP_REPORT_TTL	5.0	Silence before the headphone row goes to unknown.
RING_INTERVAL	1.5	Gap between beeps.
RING_FREQ	1000	Beep pitch in Hz. Useful range roughly 500-2000.
RING_MS	200	Beep length. Above ~400 the silence click feels delayed.

Client

Constant	Value	Effect of changing it
HEARTBEAT_INTERVAL	1.0	Rate of headphone reports and registrations.
SERVER_TIMEOUT	3.0	Missing acknowledgements before SERVER: DISCONNECTED.
DEVICE_POLL_INTERVAL	1.0	How often the audio hardware is examined.
DEVICE_REFRESH_GAP	2.0	Minimum gap between PortAudio restarts.
REPORT_REPEATS	3	Copies of a state-change report.
AUDIO_QUEUE_MAX	64	Playback backlog. Oldest packet is dropped when full.
STREAM_CLOSE_WAIT	5.0	Patience when closing a stream on an unplugged device.

10. Implementation notes

Four things in this code look odd and are load-bearing. Changing them reintroduces bugs that were awkward to find.

PortAudio must be restarted to see hardware changes

PortAudio snapshots the device list when it initialises. Without a periodic terminate-and-initialise, a headset that was unplugged is still listed forever and hot-plug is invisible. The restart never runs while a stream is open: terminating PortAudio underneath a live stream kills the process at the C level, with no traceback. An open-stream counter is held for the whole life of every stream, including the close — closing a stream on a device that was just pulled can block, and that window is exactly when a restart would be fatal.

Streams open on the tracked device, not the default

Once the headset is located by name, its indexes are used for both the capability check and the streams. Opening on the default device would silently record from the built-in microphone after a headset is unplugged.

One receive loop owns the socket

The client's socket carries heartbeats, match replies, commands and audio. A single thread calls `recvfrom` and dispatches into queues. When several threads read the same UDP socket they steal each other's datagrams at random, which produces failures that look like packet loss and never reproduce the same way twice.

Audio threads are joined before the socket closes

Closing the socket out from under a running audio thread leaves the sound card busy and raises misleading exceptions. Shutdown waits for the threads first, and if one is still closing a stream after five seconds it says so and leaves PortAudio alone.

11. Version history

Version	Contents
v1	Headphone and server-link state reporting. Persistent client socket, device tracking by name, heartbeat with acknowledgements, single receive loop.
v2	Beep on headphone connect. SHUTDOWN CLIENT with source validation. All dialogs drawn in the application theme. Watermark and layout fixes.

(c) oT — All rights reserved.